Java 8 features

Q what is the need of default method in interface?

→ The main reason default methods were introduced in java interfaces (Java 8) was backward compatibility.

The problem before Java 8

→ Suppose you have an interface

```
interface Animal {
    void eat(); // abstract method
}
```

Many classes implement it

```
class Dog implements Animal {
    public void eat() {
        Sout("Dog eats");
    }
}
```

```
class Cat implements Animal {
    @Override
    public void eat() {
        Sout("cat eats");
    }
}
```

Now imagine you want to add a new method:

```
interface Animal {
    void eat();
    void sleep();  // New method
}
```

Problem: Every existing class (Dog, cat, thousands of other) will fail to compile because they haven't implemented sleep(). This would break millions of lines of existing Java code

Default method in Interface

1. Solution is default method.:

```
public interface Bird {
    public void canFly();    //abstract method

    default int getMinimumFlyHeight() { // new method
        return 100;
    } }
public class Eagle implements Bird {
    @Override
    public void canFly() {
        // eagle fly implementation
    }
}
public class Main {
    psvm() {
        Eagle eagleObj = new Eagle();
        eagleObj.getMinimumFlyHeight();
    }
}
```

Now existing classes don't need to change.
The child class automatically inherit the default implementation.

Q. Why Default method was introduced in Java 8?

→ To add functionality in existing legacy Interface we need to use Default method.
Example stream() method in collection interface.

→ Stream() method added in ~~Java~~ and collection is inherited by so many classes if they make Stream() as abstract then they have to provide its defn in all inherited class. that's why they introduce default()

Default and Multiple Inheritance, how to handle:

```
public interface Bird {
    default boolean canBreath() {
        return true;
    }
}
```

```
public interface LivingThing {
    default boolean
        canBreath() {
            return true;
        }
}
```

```
public class Eagle implements Bird, LivingThing {
}
```

Let say we create an obj of ~~Bird~~ Eagle class

```
Eagle obj = new Eagle();
obj.canBreath();   // Now it causes ambiguity
```

Not work ✗

and compiler got confuse, whose canBreath() call here.

→ to resolve this child class has to provide the default method implementation

```
public class Eagle implements Bird, LivingThing {
    public boolean canBreath() {
        return true;
    }
}
```

```
Eagle obj = new Eagle()
obj.canBreath();
```

Now it call's eagle method implementation

Public static available in

2. Static Method (Java 8):

- We can provide the implementation of the method in interface
- But it cannot be overridden by classes which implement the interface.
- We can access it using interface name itself.
- Its by default public.

```
public interface Bird {
    static boolean canBreathe() {
        return true;
    }
}
public class Eagle implements Bird {

    public void digestiveSystemTest() {
        if (Bird.canBreathe()) {
            // do something
        }
    }
}
```

by default public →

Static method canBreathe() call with the name of interface

3. Private Method and private static method (Java 9) feature

- We can provide the implementation of method but as a private access modifier in interface.
- It brings more readability of the code. For example if multiple default method share some code, that
- It can be defined as static and non static.

- From static method, we can call only private static interface method.
- Private static method, can be called from both static and non static method.
- Private interface method can not be abstract. Means we have to provide the definition.
- It can be used inside of the particular interface only.

```
public interface Bird {
    void canFly(); // this is equivalent to
                   public abstract void canFly();
    public default void minFlyHeight(){
        myStaticPublicMethod(); // calling static method
        myPrivateMethod(); // calling private method
        myPrivateStaticMethod(); // calling private static meth
    }
    static void myStaticPublicMethod(){
        myPrivateStaticMethod(); // from static we
    }                  // can call other static method only.

    private void myPrivateMethod(){
            // private method implementation
    }
    private static void myPrivateStaticMethod(){
            // code
    }
}
```

Functional Interface and Lambda Expression

- What is Functional Interface?
- What is lambda Expression?
- How to use functional Interface with lambda expression
- Advantage of functional Interface?
- Types of functional Interface?
  - Consumer
  - Supplier
  - function
  - Predicate
- How to handle use case when functional Interface extends from other Interface (or functional Interface)

Q What is Functional Interface:

→ If an Interface contains only 1 abstract method, that is known as functional Interface.

→ Also known as SAM Interface (Single Abstract Method).

→ @FunctionalInterface keyword can be used at top of the Interface (But its optional).

```
@FunctionalInterface
public Interface Bird {
    void canFly(String val);
}
```

or

```
public Interface Bird {
    void canFly(String val);
}
```

```
@FunctionalInterface
public Interface Bird {
    void canFly(String val);
    void getHeight();
}
```

← @FunctionalInterface Annotation restrict us and throws compilation error, if we try to add more than 1 abstract method

Error / Not possible

```
@FunctionalInterface
public interface Bird {

    void canFly(String val);      // abstract method.

    default void getHeight(){     // default method
        //default method impln
    }
    Static void canEat(){         // Static method
        // code
    }
    String toString();            // object class method impln
                                  is not req, its already defined in object class.
```

Note: In FunctionalInterface, only 1 abstract method is allowed, but we can add other methods like default, static method, or Methods inherited from the Object class.

Different ways to implements the functional Interface

1) using "implements"

```
ex: @FunctionalInterface
    public interface Bird {
        void canFly(String val);
    }
    public class Eagle implements Bird {
        @Override
        public void canFly(String val){
            Sout("Eagle bird Implementation"+val);
        }
    }
Eagle eagleObj=new Eagle();
eagleObj.canFly("vertical");
```

ii) using "anonymous class"

```
public class Main {
    psvm() {
        Bird eagleObj = new Bird() {
            @override
            public void canFly(String val) {
                Sout("Eagle Bird Impl" + val);
            }
        };
        eagleObj.canFly("Vertical");
    }
}
```

anonymous class
internally a class is created with some name

iii) using "Lambda Expression"

Q what is lambda expression?

→ A lambda expression is one of the most important features introduced in Java 8. It provide a concise way to write implementations for functional interface
Think of it as "a method without a name"

General Syntax : (parameters) → expression
(lambda operator)

i) No parameter : () → expression
one line code Ex: () → sout("Hello");

ii) No parameter : () → { }
multiline code Ex: () → { sout("Hello");
return 0;
}

iii) One parameter : var → calculation
Single expn Ex: Square S = x → x*x;

iv) Multi parameter : (comma Seperated Parameters) → expression
Ex: Int sum = (a, b) → a+b;
(int a, int b) X not req to define d.T

Now see Implementation of FunctionalInterface with the help of lambda expression

iii) using lambda expression:

```
public class Main{
    psvm(){
        Bird eagleObj = (String val) ->
                        sout("Eagle Bird Implementation");
    };
    eagleObj.canFly("vertical");
    }
}
```

adv: code length reduce and easy to read code

Types of functional Interface:
present in package : java.util.function;

1. Consumer
- Represent an operation, that accept a single input parameter and returns no result.

DT is Generic type

```
@FunctionalInterface
public Interface Consumer<T>{

    void accept(T t);
}
```

ID Math.random()*100

```
public class Main{
    psvm(){
        Consumer<Integer> loggingObj = (Integer val) ->{
                if(val>10){
                    sout("Logging");
                }
        };
    }
}
```

2. Supplier
- Represent the Supplier of the result.
Accepts no input parameter but produce a result.

```
@FunctionalInterface
public Interface Supplier<T>{
        T get();
}

public class Main{
    psvm(){
        Supplier<String> eid = ()->
                "ID"+(Math.random()*100);
        sout(eid.get());
    }
}
```

3. Function
- Represent function, that accepts one argument process it and produce a result

```
@FunctionalInterface
public interface Function<T, R>{
        R apply(T t);
}
public class Main{
    psvm(){
        Function<Integer, String> integerToString =
                (Integer num) ->{
                    String res = num.toString();
                    return res;
                };
        sout(integerToString.apply(64));
    }
}
```

(Input type → T; return type → R)

4. Predicate
- Represent function, that accept one argument and return the boolean.

```
@FunctionalInterface
public interface Predicate<T>{

        boolean test(T t);
}
```

```
public class Main{
    psvm(){
        Predicate<Integer> isEven = (Integer val) -> {
            if(val%2==0){
                return true;
            }else{
                return false;
            }
        };
        sout(isEven.test(19));
    }
}
```

Note: If your requirement is not fulfil by these 4 functional Interface then you may define your own functional Interface also.